LABORATORIUM 2

TREŚCI KSZTAŁCENIA: DEFINIOWANIE KLAS I OBIEKTÓW

Spis treści

Definicja klasy 2
Operatory dostępu 4
Modyfikatory dostępu 5
Konstruktory 5
gettery settery C# 7
Zadania do samodzielnego rozwiązania 9

Programowanie obiektowe – OOP Object-Oriented Programming.

Programowanie proceduralne polega na pisaniu procedur lub metod, które wykonują operacje na danych, podczas gdy programowanie obiektowe polega na tworzeniu obiektów, które zawierają zarówno dane, jak i metody.

Programowanie obiektowe ma kilka zalet w porównaniu z programowaniem proceduralnym:

- OOP jest szybsze i łatwiejsze do wykonania
- OOP zapewnia przejrzystą strukturę dla programów
- OOP pomaga utrzymać kod C# DRY "Don't Repeat Yourself" i sprawia, że kod jest łatwiejszy w utrzymaniu, modyfikacji i debugowaniu
- OOP umożliwia tworzenie pełnych aplikacji wielokrotnego użytku z mniejszą ilością kodu i krótszym czasem programowania.

Zasada "Don't Repeat Yourself" (DRY) polega na ograniczeniu powtarzalności kodu. Tworząc kod programu należy wyodrębnić kody, które są wspólne dla aplikacji i umieścić je w jednym miejscu i ponownie wykorzystać zamiast powtarzać.

Czym jest klasa i obiekt

Definicja klasy

```
modyfikator_dostepu class nazwa_klasy
{
    // pola klasy
    modyfikator_dostepu typ_danych zmienna1;
    modyfikator_dostepu typ_danych zmienna2;
    ...
    modyfikator_dostepu typ_danych zmiennaN;
    // metody klasy
    modyfikator_dostepu zwracany_typ metoda1(lista_parametrow)
    {
        wnetrze_metody
    }
    modyfikator_dostepu zwracany_typ metoda2(lista_parametrow)
    {
        wnetrze_metody
    }
    ...
    modyfikator_dostepu zwracany_typ metodaN(lista_parametrow)
    {
        wnetrze_metody
    }
}

namespace SharpLab
{
```

```
    public class Car
    {
        //pola klasy
        public string color = "red";
        public int rok = 2023;
    }
}
```

Tworzenie obiektu

```
using SharpLab;

Car car = new Car(); // tworzenie obiektu klasy
Console.WriteLine("Samochod kolor: " + car.color + " rocznik: " + car.rok);
```

Metody:

```
namespace SharpLab
{
    public class Car
    {
        //pola klasy
        public string color = "red";
        public int rok = 2023;

        //metody
        public void view() {
            Console.WriteLine("Samochod kolor: " + color + " rocznik: " + rok);
        }
    }
}
```

```
using SharpLab;

Car car = new Car(); // tworzenie obiektu klasy
car.view();

//zmiana parametrów
car.rok = 2021;
car.color = "niebieski";
//wywołanie metody
car.view();
```

Konstruktor

```
namespace SharpLab
{
    public class Car
    {
        //pola klasy
        public string color;
        public int rok;

        //metody

        //konstruktory
        // konstruktor  domyślny bezparametrowy
        public Car() {
            color = "red";
            rok= 100;
        }
        //konstruktor z parametrami
        public Car(string color) {
```

```
            this.color = color;
        }
        public Car(string color, int rok) {
            this.color = color;
            this.rok = rok;
        }

        public void view() {
            Console.WriteLine("Samochod kolor: " + color + " rocznik: " + rok);
        }
    }
}

using SharpLab;
Car car = new Car(); // tworzenie obiektu klasy
car.view();

Car car1 = new Car("niebieski");
car1.view();
Car car2 = new Car("czarny", 2020);
car2.view();


//tworzenie tablicy obiektów i ich wyświetlanie
// I sposoób
Car[] arrayCar = new Car[5];

arrayCar[0] = new Car("Czerwony", 2023);
arrayCar[1] = new Car("Niebieski", 2021);
arrayCar[2] = new Car("Zielony", 1999);
arrayCar[3] = new Car("Czarny", 2015);
arrayCar[4] = new Car("Niebieski", 2019);

foreach (Car item in arrayCar)
{
    item.view();
}

// II sposób
Car[] arrayCar2 = new Car[] {
    new Car("Czerwony", 2023),
    new Car("Niebieski", 2021),
    new Car("Zielony", 1999)
};

//III sposob
Car[] cars = new Car[3];

string[] color = { "Niebieski", "Czerwony", "Czarny" };
int[] rok = {2023, 2022,2021 };

for (int i = 0; i < cars.Length; i++)
{
    cars[i] = new Car(color[i], rok[i]);
}
```

Operatory dostępu

C# oferuje kilka operatorów dostępu, które służą do manipulowania członkami klas (polami, właściwościami, metodami).

- . (kropka) - Operator dostępu do elementów klasy/obiektu.

- this - Służy do odwołania się do bieżącej instancji klasy (obiektu).

Modyfikatory dostępu

- public: Dostęp z dowolnego miejsca w programie.
- private: Dostęp tylko wewnątrz tej samej klasy.
- protected: Dostęp wewnątrz klasy i klas pochodnych.
- internal: Dostęp w obrębie tego samego zestawu (assembly).

Konstruktory

W C# istnieje kilka rodzajów konstruktorów, które można zdefiniować w klasie w zależności od potrzeb, wśród nich wyróżniamy konstruktor:

- domyślny,
- parametryczny,
- kopiujący,
- statyczny,
- prywatny,
- bazowy.

Każdy z tych konstruktorów służy różnym celom i pozwala na elastyczną inicjalizację obiektów w różnych scenariuszach programistycznych.

1. Konstruktor domyślny

Konstruktor domyślny to konstruktor, który nie przyjmuje żadnych argumentów i jest automatycznie generowany przez kompilator, jeśli nie zdefiniujesz żadnego innego konstruktora w klasie. Można go także zdefiniować ręcznie.

```
public class Car
{
    public Car()  // Konstruktor domyślny
    {
        Console.WriteLine("Domyślny konstruktor został wywołany.");
    }
}
```

2. Konstruktor parametryczny

Konstruktor parametryczny przyjmuje argumenty, które są wykorzystywane do inicjalizacji obiektu. Pozwala to na przypisanie wartości polom lub właściwościom obiektu już w momencie tworzenia instancji.

```
public class Car
{
    public string Marka;
    public string Model;

    public Car(string marka, string model)  // Konstruktor parametryczny
    {
        Marka = marka;
        Model = model;
    }
}
```

3. Konstruktor kopiujący

Konstruktor kopiujący służy do tworzenia nowego obiektu na podstawie już istniejącego obiektu tego samego typu. Umożliwia to klonowanie obiektów.

```
public class Car
{
    public string Marka;
    public string Model;

    // Konstruktor parametryczny
    public Car(string marka, string model)
    {
        Marka = marka;
        Model = model;
    }

    // Konstruktor kopiujący
    public Car(Car innySamochod)
    {
        Marka = innySamochod.Marka;
        Model = innySamochod.Model;
    }
}
```

4. Konstruktor statyczny

Konstruktor statyczny jest wywoływany tylko raz dla klasy (nie dla obiektu) i służy do inicjalizacji pól statycznych tej klasy. Nie przyjmuje żadnych argumentów i nie można go wywołać ręcznie.

```
public class Car
{
    public static int LiczbaSamochodow;

    // Konstruktor statyczny
    static Car()
    {
        LiczbaSamochodow = 0;
        Console.WriteLine("Konstruktor statyczny został wywołany.");
    }

    public Car()
    {
        LiczbaSamochodow++;
    }
}
```

5. Konstruktor prywatny

Konstruktor prywatny nie może być używany do tworzenia instancji obiektu na zewnątrz klasy. Jest przydatny w implementacji wzorców projektowych, takich jak Singleton, gdzie chcemy kontrolować liczbę instancji klasy.

```
public class Singleton
{
    private static Singleton instancja;

    // Prywatny konstruktor
    private Singleton()
    {
    }
```

```
    public static Singleton PobierzInstancje()
    {
        if (instancja == null)
        {
            instancja = new Singleton();
        }
        return instancja;
    }
}
```

6. Konstruktor bazowy (wywołanie konstruktora klasy bazowej)

Konstruktor klasy pochodnej może wywoływać konstruktor klasy bazowej przy użyciu słowa kluczowego base. Pozwala to na inicjalizację pól klasy bazowej.

```
public class Pojazd
{
    public Pojazd()
    {
        Console.WriteLine("Konstruktor klasy bazowej Pojazd.");
    }
}

public class Samochod : Pojazd
{
    public Samochod() : base()  // Wywołanie konstruktora bazowego
    {
        Console.WriteLine("Konstruktor klasy pochodnej Samochod.");
    }
}
```

7. Konstruktor z inicjalizacją właściwości

Konstruktor może być użyty w połączeniu z inicjalizacją właściwości obiektu, aby zainicjalizować niektóre lub wszystkie pola/właściwości obiektu.

```
public class Car
{
    public string Marka { get; set; }
    public string Model { get; set; }

    public Car() { }

    public Car(string marka) : this()  // Konstruktor z inicjalizacją
    {
        Marka = marka;
    }
}
```

gettery settery C#

W C#, słowa kluczowe get i set są używane do definiowania tzw. właściwości (properties) w klasach. Właściwości pozwalają na kontrolowany dostęp do pól prywatnych klasy, umożliwiając ich odczyt lub modyfikację. Są one często używane do enkapsulacji (ukrycia) wewnętrznych danych, oferując jednocześnie interfejs do zarządzania nimi.

Składnia właściwości:

- get: Używany do pobierania wartości właściwości.
- set: Używany do ustawiania wartości właściwości.

```
public class Car
{
    // Pole prywatne
    private string marka;

    // Właściwość z get i set
    public string Marka
    {
        get
        {
            return marka;  // Zwraca wartość pola 'marka'
        }
        set
        {
            marka = value;  // Ustawia wartość pola 'marka'
        }
    }
}
```

W tym przykładzie właściwość Marka pozwala na uzyskiwanie i ustawianie wartości pola marka, które jest prywatne. Dzięki temu możliwa jest kontrola nad dostępem do tego pola z zewnątrz.

```
Car samochod = new Car();
samochod.Marka = "Toyota";  // Użycie settera
Console.WriteLine(samochod.Marka);  // Użycie gettera
```

Zadania do samodzielnego rozwiązania

Zadanie 1.

Napisz klasę Osoba, która będzie przechowywać informacje o imieniu, nazwisku oraz wieku osoby.

- Zaimplementuj konstruktor, który będzie przyjmował wszystkie trzy wartości.
- Użyj właściwości Imie, Nazwisko, Wiek, z walidacją:
  - Imię i Nazwisko muszą mieć co najmniej 2 znaki.
  - Wiek musi być liczbą dodatnią.
- Zaimplementuj metodę WyswietlInformacje(), która wyświetli informacje o osobie.

Zadanie 2.

Napisz klasę BankAccount, która będzie symulowała konto bankowe.

- Zaimplementuj właściwości Saldo (publiczne, tylko do odczytu) i Wlasciciel.
- Dodaj metodę Wplata(decimal kwota), która pozwala na zwiększenie salda, oraz metodę Wyplata(decimal kwota), która sprawdzi, czy jest wystarczająca ilość środków, a następnie odejmie odpowiednią kwotę.
- Użyj operatorów dostępu, aby zabezpieczyć saldo przed bezpośrednią modyfikacją.

Przykład użycia:

```
BankAccount konto = new BankAccount("Jan Kowalski", 1000);
konto.Wplata(500);
konto.Wyplata(200);
Console.WriteLine($"Saldo: {konto.Saldo}");
```

Zadanie 3.

Napisz klasę Student, która będzie przechowywała imię, nazwisko i tablicę ocen.

- Zaimplementuj właściwość SredniaOcen, która obliczy i zwróci średnią ocen.
- Dodaj metodę DodajOcene(int ocena), która doda nową ocenę do tablicy.
- Zaimplementuj konstruktor inicjujący imię i nazwisko studenta.

Zadanie 4.

Stwórz klasę Licz z:

- publicznym polem value przechowującym wartość liczbową.
- metodą Dodaj przyjmującą jeden parametr i dodającą przekazaną wartość do wartości trzymanej w polu value.
- analogiczną operację odejmij

W Main utwórz kilka obiektów klasy Licz i wykonaj różne operacje.

Do klasy Licz dodaj konstruktor z jednym parametrem - który inicjuje pole wartość na liczbę przekazaną w parametrze.

Zmień widoczność pola na private i dodaj funkcję wypisującą stan obiektu (pole value).

Zadanie 5.

Stwórz klasę Sumator z:

- publicznym polem Liczby będącym tablicą liczb
- metodą Suma zwracającą sumę liczb z pola Liczby
- metodę SumaPodziel2 zwracającą sumę liczb z tablicy, które są podzielne przez 2

Zmień widoczność pola Liczby na private oraz dodaj konstruktor.

Dodaj metodę: int IleElementów () zwracającej liczbę elementów na w tablicy

Dodaj metodę wypisującą wszystkie elementy tablicy

Dodaj metodę przyjmującą dwa parametry: lowIndex oraz highIndex, która wypisze elementy o indeksach >= lowIndex oraz <= highIndex. Metoda powinna zadziałać poprawnie, gdy lowIndex lub highIndex wykraczają poza zakres tablicy (pominąć te elementy).